



Python基本数据类型-字符串

- 概念：字符串是字符的序列表示
- 表示：用单引号(')或者双引号("")或者三引号('''')括起来的一个或多个字符
 - 单引号字符串：双引号可以作为字符串的一部分
 - 双引号字符串：单引号可以作为字符串的一部分
 - 三引号字符串：单引号/双引号均可作为字符串的一部分



字符串类型

单引号字符串:

```
python

s1 = 'Hello, Python!'
s2 = 'He said "yes"'      # 双引号直接嵌入
```

双引号字符串:

```
python

s3 = "Hello, Python!"
s4 = "It's a nice day"    # 单引号直接嵌入
```

三引号字符串:

```
python

s5 = '''He said "It's OK"'''    # 单双引号均可嵌入

s6 = """
第一行
第二行
第三行
"""                                # 支持跨多行书写
```



字符串类型

■ 单引号、双引号、三引号汇总

引号类型	内嵌单引号	内嵌双引号	跨行
'...'	需转义 \'	直接写	不支持
"..."	直接写	需转义 \"	不支持
'...' / '...' / '...' / '...'	直接写	直接写	支持

字符串类型

■ 转义符与原始字符串

- Python 转义符：\ (反斜杠)
 - 用于输出特殊字符

```
>>> print('It\'s OK')
It's OK
>>> print("He said \"Hi\"")
He said "Hi"
```

- 原始字符串
 - 引号前加 **r** , \ 不再转义

```
>>> print("C:\\Users\\test")
C:\Users\test
>>> print(r"C:\Users\test")
C:\Users\test
```

格式化控制字符

■ 转义符号汇总

格式化控制字符	含义
<code>\n</code>	换行，光标移动到下行行首
<code>\t</code>	水平制表
<code>\\</code>	反斜杠本身
<code>\'</code>	单引号
<code>\"</code>	双引号
<code>\r</code>	回车，光标移动到本行行首
<code>\0</code>	NULL，什么也没有
<code>\b</code>	回退，向后退一格
<code>\a</code>	蜂鸣，响铃
<code>\f</code>	换页
<code>\v</code>	垂直制表

格式化控制字符 \n 与 \t

- 输出特殊效果，表达一些不可直接打印的信息

- \n 换行，光标移动到下行行首

```
>>> print("Hello\nWorld\n\nGoodbye 32")
```

```
Hello          # \n 换到下一行
```

```
World          # \n\n 连续两个换行，产生一个空行
```

```
Goodbye 32
```

- \t 水平制表，常用于对齐输出

⚠ 左侧不用 \t 时名字长度不同导致后面的列错位；右侧用了 \t 后自动对齐

```
>>> print("张三 20 北京")
张三 20 北京
>>> print("李四 22 上海")
李四 22 上海
>>> print("王五五 19 广州")
王五五 19 广州
```

```
>>> print("张三\t20\t北京")
张三      20      北京
>>> print("李四\t22\t上海")
李四      22      上海
>>> print("王五五\t19\t广州")
王五五    19      广州
```

字符串索引

■ 两种序号体系：正向递增和反向递减

- 字符串中的编号叫做“索引”
- 字符串最左端位置标记为0，依次增加
- 从最右端第一个字符是-1，向左依次递减



字符串索引

■ 单个索引辅助访问字符串中的特定位置

格式为 <string>[<索引>]



```
>>> tempStr = "Hello World"
```

```
>>> print(tempStr[1])      # 正向索引, 第2个字符 (从0开始)
```

e

```
>>> print(tempStr[-10])   # 反向索引, 倒数第10个字符
```

e



字符串索引

- 可以通过两个索引值确定一个位置范围，返回这个范围的子串

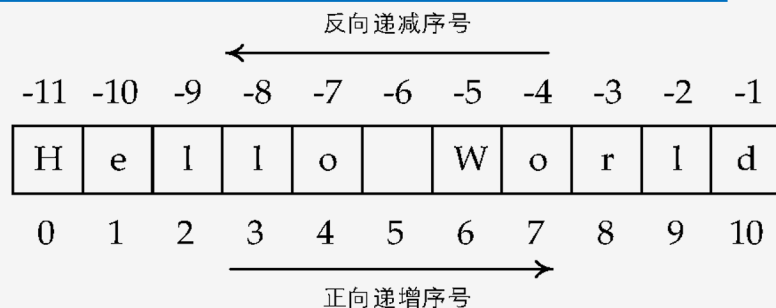
格式： `<string>[<start>:<end>]`

- start 和end 都是整数型数值，这个子序列从索引start开始直到索引end结束，但不包括end位置

```
>>> tempStr = 'Hello world'
>>> print(tempStr[1:5])    # 切片：取索引1到4的字符（包头不包尾）
ello
```

字符串索引

练习



```
>>> str = "Hello World"
```

```
>>> print(str)
```

完整字符串

```
Hello World
```

```
>>> print(str[:])
```

省略首尾，等同于完整字符串

```
Hello World
```

```
>>> print(str[0])
```

第一个字符

```
H
```

```
>>> print(str[2:5])
```

索引2到4

```
llo
```

```
>>> print(str[0:-1])
```

索引0到倒数第二个

```
Hello Worl
```

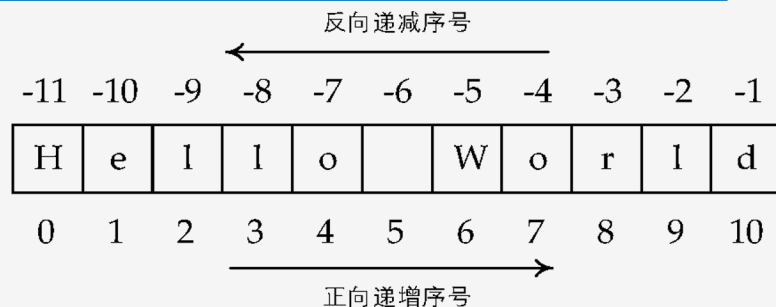
```
>>> print(str[:-1])
```

同上，省略0

```
Hello Worl
```

字符串索引

练习



```
>>> print(str[2:])
```

```
llo World
```

```
>>> print(str[-2:-5])
```

索引2到末尾

start > end, 切不到内容

(空字符串)

```
>>> print(str[-5:-2])
```

倒数第5到倒数第3

```
Wor
```

```
>>> print(str[-2:4])
```

-2是索引9, 4<9, 切不到内容

(空字符串)

```
>>> print(str[4:-2])
```

索引4到倒数第3

```
o Wor
```

■ 增加步长

- 格式： `<string>[<start>:<end>:<step>]`
- start 和 end 都是整数型数值，子序列从索引 start 开始直到索引 end 结束，但不包括 end 位置
- step 为步长，默认为 1

```
>>> tempStr = 'Hello world'
```

```
>>> print(tempStr[0:5:2])      # 步长为2，每隔一个取一个  
Hlo
```

```
>>> print(tempStr[::3])        # 步长为3，每隔两个取一个  
Hlwl
```

```
>>> print(tempStr[::-1])        # 步长为-1，反转字符串  
dlrow olleH
```

字符串索引

■ 练习题

```
python
```

```
s = "Python Programming"
```

请写出以下操作的结果：

1. `s[0]`
2. `s[-1]`
3. `s[7:18]`
4. `s[:6]`
5. `s[-3:-6]`
6. `s[::2]`
7. `s[::-1]`

正向	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17
	P	y	t	h	o	n	空格	P	r	o	g	r	a	m	m	i	n	g
反向	-18	-17	-16	-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1

■ 练习题

```
>>> s = "Python Programming"
>>> print(s[0])          # P
P
>>> print(s[-1])         # g
g
>>> print(s[7:18])       # Programming
Programming
>>> print(s[:6])         # Python
Python
>>> print(s[-3:-6])      # start 在 end 之后且步长为正，结果为空字符串

>>> print(s[::2])        # Pto rgamn
Pto rgamn
>>> print(s[::-1])       # gnimmargorP nohtyP
gnimmargorP nohtyP
```

基本的字符串操作符

操作符	含义
<code>x + y</code>	连接两个字符串 x 和 y
<code>x * n</code> 或 <code>n * x</code>	复制 n 次字符串 x
<code>x in s</code>	如果 x 是 s 的子串，返回 True，否则返回 False
<code>x not in s</code>	如果 x 不是 s 的子串，返回 True，否则返回 False

```
>>> "Hello" + " World"           # 连接
'Hello World'
>>> "Ha" * 3                      # 复制
'HaHaHa'
>>> "llo" in "Hello"              # 判断子串
True
>>> "abc" not in "Hello"          # 判断不包含
True
```



基本的字符串操作符

- ⚠ Python的字符串不能被改变 (不可变类型)

```
>>> s = "Hello"
```

```
>>> s[0] = "m"
```

```
TypeError: 'str' object does not support item assignment
```

内置的字符串处理函数

- **函数**：独立调用，把字符串作为参数传进去

函数	含义
<code>len(x)</code>	返回字符串的长度
<code>str(x)</code>	返回任意类型 x 对应的字符串形式
<code>int(x)</code>	将字符串 x 转为整数
<code>float(x)</code>	将字符串 x 转为浮点数
<code>chr(x)</code>	返回 Unicode 编码 x 对应的单字符
<code>ord(x)</code>	返回单字符 x 表示的 Unicode 码
<code>hex(x)</code>	返回整数 x 对应的十六进制小写形式字符串
<code>oct(x)</code>	返回整数 x 对应的八进制小写形式字符串
<code>bin(x)</code>	返回整数 x 对应的二进制形式字符串
<code>type(x)</code>	返回 x 的类型

内置的字符串处理函数

```
>>> len("Hello")           # 长度
5

>>> str(123)               # 数字转字符串
'123'

>>> int("456")             # 字符串转整数
456

>>> chr(65)                # Unicode码 → 字符
'A'

>>> ord('A')               # 字符 → Unicode码
65

>>> hex(255)               # 十六进制
'0xff'

>>> oct(255)               # 八进制
'0o377'

>>> bin(255)               # 二进制
'0b11111111'
```



内置的字符串处理方法

■ 方法：通过字符串对象用 `.` 调用，属于字符串自带的功能

一、大小写转换

方法	描述
<code>str.lower()</code>	返回 str 的副本，全部字符小写
<code>str.upper()</code>	返回 str 的副本，全部字符大写
<code>str.title()</code>	返回 str 的副本，每个单词首字母大写
<code>str.capitalize()</code>	返回 str 的副本，仅首字母大写

```
>>> s = "hello world"
>>> s.lower()           # 全部小写
'hello world'
>>> s.upper()          # 全部大写
'HELLO WORLD'
>>> s.title()          # 每个单词首字母大写
'Hello World'
>>> s.capitalize()     # 仅第一个字母大写
'Hello world'
```

⚠ 返回新字符串，不会修改原字符串
(字符串不可变)

```
>>> s.upper()
'HELLO WORLD'
>>> s                                     # s 还是原来的值
'hello world'
```

内置的字符串处理方法

■ 方法：通过字符串对象用 `.` 调用，属于字符串自带的功能

方法	描述
<code>str.islower()</code>	所有区分大小写的字符都是小写，且至少含一个此类字符
<code>str.isupper()</code>	所有区分大小写的字符都是大写，且至少含一个此类字符
<code>str.isdigit()</code>	所有字符都是数字字符（含 0-9 及 ² 等上标数字）
<code>str.isnumeric()</code>	所有字符都是数值字符（含中文数字、罗马数字、 $\frac{1}{2}$ 等）
<code>str.isalpha()</code>	所有字符都是字母（含汉字等 Unicode 文字）
<code>str.isalnum()</code>	所有字符都是字母或数字
<code>str.isspace()</code>	所有字符都是空白字符（空格、 <code>\t</code> 、 <code>\n</code> 等）
<code>str.isprintable()</code>	所有字符都是可打印的，空字符串也返回 True
<code>str.startswith(prefix[, start[, end]])</code>	在 <code>[start, end)</code> 范围内是否以 <code>prefix</code> 开头， <code>prefix</code> 可为元组
<code>str.endswith(suffix[, start[, end]])</code>	在 <code>[start, end)</code> 范围内是否以 <code>suffix</code> 结尾， <code>suffix</code> 可为元组

内置的字符串处理方法

■ 方法：通过字符串对象用 `.` 调用，属于字符串自带的功能

```
>>> "abc123".islower()
True
# 数字不区分大小写，被忽略

>>> "123".islower()
False
# 没有区分大小写的字符

>>> "ABC123".isupper()
True

>>> "123".isdigit()
True

>>> "²".isdigit()
True
# 上标数字也算

>>> "二十".isnumeric()
True
# 中文数字

>>> "½".isnumeric()
True
# 分数

>>> "二十".isdigit()
False
# isdigit 不识别中文数字

>>> "abc".isalpha()
True

>>> "中文".isalpha()
True
# 汉字也算字母
```



内置的字符串处理方法

- **方法**：通过字符串对象用 `.` 调用，属于字符串自带的功能

```
>>> "abc123".isalnum()
True
>>> "abc 123".isalnum()                # 含空格
False
>>> "   ".isspace()
True
>>> "\n\t".isspace()                   # 换行、制表符也属于空白字符
True
>>> "Hello\n".isprintable()             # \n 不可打印
False
>>> "".isprintable()                    # 空字符串返回 True
True
>>> "".isalpha()                        # 其余 is 系列方法对空字符串返回 False
False
```

⚠ 空格 `' '` 既是空白字符，也是可打印字符；
而 `\t`、`\n` 是空白字符，但不可打印



内置的字符串处理方法

- **方法**：通过字符串对象用 `.` 调用，属于字符串自带的功能

```
>>> "hello.py".startswith("hello")
True
>>> "hello.py".startswith("llo", 2)           # 判断范围为 "llo.py"
True
>>> "hello.py".endswith(".py")
True
>>> "hello.py".endswith("hello", 0, 5)        # 判断范围为 "hello"
True
>>> "hello.py".endswith((".py", ".txt"))      # 元组中任意一个匹配即返回 True
True
```

内置的字符串处理方法

■ 方法：通过字符串对象用 . 调用，属于字符串自带的功能

三、查找与计数

方法	描述
<code>str.find(sub[, start[, end]])</code>	返回 sub 第一次出现的索引，找不到返回 -1
<code>str.index(sub[, start[, end]])</code>	同 find 找不到时报错 ValueError
<code>str.count(sub[, start[, end]])</code>	返回 sub 出现的次数

```
>>> s = "Hello World"
>>> s.find("World")          # 找到，返回起始索引
6
>>> s.find("Python")        # 找不到，返回-1
-1
>>> s.index("World")         # 找到，返回起始索引
6
>>> s.index("Python")        # 找不到，报错！
ValueError: substring not found
>>> "abcabcabc".count("abc")  # 统计出现次数
3
```

⚠ **find() 和 index()**
功能相同，区别在于找不到时 find() 返回 -1，index() 直接报错

内置的字符串处理方法

■ **方法**：通过字符串对象用 `.` 调用，属于字符串自带的功能

四、替换与分割

方法	描述
<code>str.replace(old, new[, count])</code>	将 old 替换为 new, count 指定最多替换次数
<code>str.split(sep=None, maxsplit=-1)</code>	按 sep 分割，返回列表
<code>str.join(iterable)</code>	用 str 连接 iterable 中的每个元素

iterable 指**可迭代对象**，简单理解就是
"能用 for 循环逐个取出元素的对象"

内置的字符串处理方法

- **方法**：通过字符串对象用 `.` 调用，属于字符串自带的功能

```
>>> "Hello World".replace("World", "Python")
'Hello Python'
>>> "aaa".replace("a", "b", 2)           # 最多替换2次
'bba'
```

```
>>> "A,B,C,D".split(",")                # 按逗号分割
['A', 'B', 'C', 'D']
>>> "Hello World".split()                # 默认按空格分割
['Hello', 'World']
>>> "A,B,C,D".split(",", 2)              # 最多分割2次
['A', 'B', 'C,D']
```

⚠ `split()` 和 `join()` 互为反操作

```
>>> ",".join(["A", "B", "C"])            # 用逗号连接列表
'A,B,C'
>>> " ".join(["Hello", "World"])         # 用空格连接
'Hello World'
```

内置的字符串处理方法

- **方法**：通过字符串对象用 `.` 调用，属于字符串自带的功能

五、去除与填充

方法	描述
<code>str.strip([chars])</code>	去掉左侧和右侧的指定字符
<code>str.lstrip([chars])</code>	只去掉左侧的指定字符
<code>str.rstrip([chars])</code>	只去掉右侧的指定字符
<code>str.center(width[, fillchar])</code>	字符串居中，用 <code>fillchar</code> 填充至 <code>width</code> 宽度
<code>str.zfill(width)</code>	左侧填充 0 至 <code>width</code> 宽度

内置的字符串处理方法

- **方法**：通过字符串对象用 `.` 调用，属于字符串自带的功能

```
>>> "  Hello  ".strip()           # 去掉两侧空格
'Hello'
>>> "  Hello  ".lstrip()          # 只去左侧
'Hello  '
>>> "  Hello  ".rstrip()          # 只去右侧
'  Hello'
>>> "xxHelloxx".strip("x")        # 去掉两侧指定字符
'Hello'

>>> "Hi".center(10, "-")          # 居中，用-填充
'----Hi----'
>>> "42".zfill(6)                 # 左侧补0
'000042'
```

⚠ `strip()` 不传参数时默认去除空格、`\n`、`\t` 等空白字符，不只是空格

字符串操作符、处理函数、处理方法练习题

```
password = " Hello2026Python! "
```

请完成以下操作：

1. 去掉密码两侧的空格，赋值给 `pwd`
2. 输出清洗后的密码和它的长度
3. 检测密码中是否包含 `"2026"`
4. 检测密码是否以 `"Hello"` 开头
5. 检测密码是否以 `"!"` 结尾
6. 统计字母 `"o"` 出现了几次
7. 把密码全部转为大写
8. 把密码中的 `"2026"` 替换为 `"****"`
9. 用 `"-"` 把密码每个字符隔开
10. 密码是否全是字母？全是数字？
11. 找到 `"Python"` 在密码中的起始位置
12. 把密码反转输出

字符串操作符、处理函数、处理方法练习题

1. 去掉密码两侧的空格，赋值给 `pwd`
2. 输出清洗后的密码和它的长度

python

```
>>> pwd = password.strip()
>>> print(pwd)
Hello2026Python!
>>> print(len(pwd))
16
```

3. 检测密码中是否包含 `"2026"`
4. 检测密码是否以 `"Hello"` 开头
5. 检测密码是否以 `"!"` 结尾
6. 统计字母 `"o"` 出现了几次

python

```
>>> "2026" in pwd
True
>>> pwd.startswith("Hello")
True
>>> pwd.endswith("!")
True
>>> pwd.count("o")
2
```

字符串操作符、处理函数、处理方法练习题

7. 把密码全部转为大写
8. 把密码中的 "2026" 替换为 "****"
9. 用 "-" 把密码每个字符隔开

python

```
>>> pwd.upper()
'HELLO2026PYTHON!'
>>> pwd.replace("2026", "****")
'Hello****Python!'
>>> "-".join(pwd)
'H-e-l-l-o-2-0-2-6-P-y-t-h-o-n-!'
```

10. 密码是否全是字母？全是数字？
11. 找到 "Python" 在密码中的起始位置
12. 把密码反转输出

python

```
>>> pwd.isalpha()
False
>>> pwd.isdigit()
False
>>> pwd.find("Python")
8
>>> pwd[::-1]
'!nohtyP6202olleH'
```